

**De la Intuición a la Ingeniería: La Crónica de una Transformación
Arquitectónica Basada en Evidencia para un Ecosistema de Salud Digital**

Juan David Artunduaga Gómez
Servicio Nacional de Aprendizaje (SENA)
Neiva
Colombia

Nota del Autor

ORCID : 0000-0000-0000-0000. Correspondencia: artunduagajuan21@gmail.com

Resumen

Abstract

El presente documento expone la evolución técnica, conceptual y humana del proyecto *Appointments-Project-Back*, un sistema de gestión de citas médicas desarrollado en .NET 9, SQL Server, Docker y Angular, cuyo propósito es optimizar la programación de servicios de salud en entornos municipales. A partir del análisis crítico de veinte artículos científicos recientes, se abordan los principales desafíos enfrentados durante el desarrollo: la necesidad de una arquitectura limpia y modular, la importancia de la interoperabilidad basada en estándares como FHIR, el rol de la usabilidad en poblaciones adultas mayores, la toma de decisiones sobre infraestructura sostenible, y la implementación de un modelo de seguridad orientado al riesgo real de los datos médicos.

El documento describe el proceso de reconstrucción interna del backend, pasando de un diseño monolítico rígido a un monolito modular mantenible, con separación de responsabilidades utilizando principios de Clean Architecture y Domain-Driven Design. Además, se expone el rediseño de los DTOs para lograr compatibilidad semántica con FHIR sin sacrificar la estructura actual de la base de datos.

En términos de experiencia de usuario, se detalla la adaptación de la interfaz para mejorar la accesibilidad de adultos mayores mediante tipografía adaptable, alto contraste, patrones lineales de navegación y retroalimentación visual clara. Asimismo, se justifican decisiones de infraestructura, descartando microservicios innecesarios en favor de un despliegue eficiente con Docker, optimizaciones energéticas mediante WebSockets y un enfoque sostenible para servicios de salud locales.

Finalmente, se presenta un análisis profundo del modelo de seguridad, incorporando autenticación JWT, autorización por roles, auditoría, principios Zero Trust y mitigación activa de riesgos. El resultado es un sistema robusto, accesible y escalable, fundamentado en evidencia científica y diseñado con empatía hacia los usuarios y responsabilidad hacia los datos que gestiona.

Palabras Claves: buenas prácticas, ingeniería de software, investigación aplicada, reproducibilidad

De la Intuición a la Ingeniería: La Crónica de una Transformación Arquitectónica Basada en Evidencia para un Ecosistema de Salud Digital

Introducción: Cuando Programar No es Suficiente

Si alguien me hubiera dicho hace unos meses que terminaría escribiendo un documento de veinte páginas sobre cómo reconstruí mi propio proyecto desde cero, le habría soltado una carcajada. Mi mentalidad en aquel entonces era simple: si compila y no saca errores, está bien. No tenía ni la más remota idea de cuán equivocado estaba.

Mi proyecto de grado, un sistema de gestión de citas médicas desarrollado en .NET 9, SQL Server, Docker y Angular, nació como nace la mayoría de proyectos de estudiantes: con prisa, con motivación, y con un profundo desconocimiento de lo que significa hacer software para el mundo real. En mis primeras versiones, tenía un backend que respondía, un frontend que pintaba pantallas bonitas, y una base de datos que guardaba lo que le echara encima. En mi cabeza, eso era suficiente para decir que el proyecto estaba casi listo para producción”.

Pero una noche —siempre es de noche cuando llegan las revelaciones— mientras miraba mi archivo Program.cs lleno de servicios que yo mismo no entendía, y un controlador CitationController.cs que parecía un Frankenstein digital, me golpeó la pregunta que cambió el rumbo de todo:

”¿Y si una persona REAL usa esto para una urgencia médica... qué pasa si algo sale mal?”

Ahí se me fue la luz interna, como cuando descubres que has construido una casa bonita en un terreno arenoso. Nada estaba verdaderamente mal... pero nada estaba verdaderamente bien.

Mis DTOs eran inconsistentes.

Mis entidades EF Core eran un desastre semántico.

Tenía validaciones repetidas en tres capas distintas.

La palabra “arquitectura” para mí significaba carpetas ordenadas”.

Y mi idea de seguridad era agregar [Authorize] y rezar.

Entonces tomé la decisión más difícil del proyecto: detener todo avance, romper lo que ya había hecho y comenzar a estudiar en serio.

No para aprender a programar —eso ya lo hacía— sino para aprender a hacer ingeniería.

Porque en software, programar y hacer ingeniería no son la misma cosa.

La lectura que me cambió el rumbo

Durante dos semanas me dediqué a leer artículos científicos, papers académicos y estudios recientes sobre: • arquitecturas limpias • interoperabilidad FHIR • usabilidad para adultos mayores • seguridad Zero Trust • microservicios vs monolitos • rendimiento en backend • pruebas de software con IA

Leí sobre AppCraft, FHIR, Clean Architecture, pruebas visuales con IA, SDN, Edge Computing,DDD...

Y cada artículo era otro golpe a mi ego.

Pero, al mismo tiempo, cada artículo me ofrecía un camino.

Fue como si un montón de ingenieros del mundo real me estuvieran diciendo: "Lo que construiste funciona... pero no se puede mantener, no se puede auditar, no se puede escalar y no se puede confiar."

Y realmente, cuando manejas datos de salud, no puedes darte el lujo de fallar.

El choque entre la teoría y mi propio código

Lo más duro no fue leer lo que decían los expertos.

Lo más duro fue compararlo con lo que yo había hecho.

Por ejemplo: • En mi controlador de personas (PersonController), mezclaba lógica de negocio, validaciones, mapeos manuales y queries. Todo en un mismo archivo. • Mis servicios como PersonService devolvían entidades completas cuando deberían devolver DTOs planos y seguros. • Mi base de datos tenía nombres como tbl_clientes, regCita, tipoDocTbl, completamente alejados de cualquier estándar médico mundial. • Mis

modelos EF Core no seguían reglas DDD, ni principios de agregados, ni consistencia semántica. • Exponía IDs internos sin pensarlo dos veces. • Tenía APIs que devolvían respuestas diferentes según cómo me sintiera ese día”.

Mirando ese código con ojos nuevos, me di cuenta de que sí, funcionaba... pero no era ingeniería.

El contraste emocional: orgullo vs responsabilidad

Durante el proceso, experimenté un choque emocional fuerte.

Por un lado, sentía orgullo. Había levantado una API, configurado Docker, creado una arquitectura por capas, conectado Angular... Eso no es poca cosa para un estudiante.

Pero por otro lado, sentía responsabilidad. Este no era un proyecto cualquiera. No era un e-commerce, ni una red social escolar. Era un sistema de salud. Y en sistemas de salud: • un error de autenticación es grave, • un fallo de seguridad es devastador, • una mala UX puede impedir que un paciente pida una cita, • un mal diseño puede retrasar un diagnóstico, • una decisión de arquitectura puede impactar la vida de una persona.

Ese fue el punto en el que dejé de ser un estudiante emocionado por tener pantallas bonitas y pasé a ser alguien consciente del impacto de su trabajo.

Entender que «construir software» no es «hacer pantallas»

Aprendí, golpe tras golpe, que los sistemas profesionales no se construyen con: • trucos de StackOverflow • vídeos de YouTube • copiar controladores de tutoriales • rellenar archivos como Startup.cs sin entender • repetir patrones que no comprendo

Se construyen con decisiones informadas. Con principios sólidos. Con estándares que trascienden lenguajes. Con pruebas. Con responsabilidad. Y, sobre todo, con humildad técnica.

Decidí reconstruir mi sistema desde adentro

Después de leer 20 artículos académicos, confirmé algo: Mi sistema no estaba mal... simplemente estaba verde.

Así que tomé una decisión radical:

- Reescribir mis controladores para que cumplieran responsabilidad única
- Migrar mis DTOs hacia estructuras estandarizadas y compatibles con FHIR
- Alinear mi dominio con el lenguaje del negocio (DDD)
- Mejorar mi capa de servicios para separar lógica de orquestación y lógica de negocio
- Aplicar seguridad Zero Trust en cada endpoint
- Optimizar el backend para reducir carga y mejorar sostenibilidad
- Rediseñar pantallas para adultos mayores
- Preparar el sistema para interoperabilidad futura
- Documentar flujos como si trabajara en un equipo profesional

Ese fue el verdadero comienzo del proyecto. No cuando escribí mi primer controlador. No cuando hice el primer Docker Compose. No cuando levanté SQL Server en contenedor. El verdadero proyecto comenzó cuando entendí que el software no se programa: se diseña.

Arquitectura: Limpiando el Desastre que Nadie Veía

(Versión ampliada y profundamente integrada con tu backend real)

Cuando comencé mi proyecto, yo estaba convencido de que la arquitectura era simplemente “ordenar carpetas”. Si tenía una carpeta para Controllers, otra para Services, otra para Models, y otra para DTOs, eso ya significaba —según yo— que mi proyecto era “arquitectura por capas”.

Nada más lejos de la realidad.

Durante las primeras semanas, mi backend parecía estable. No crasheaba, respondía rápido, y EF Core hacía lo que yo quería. Pero cuando leí artículos como el de Mulla, 2024 sobre Clean Architecture y comparé su propuesta con mi propia solución... sentí esa incomodidad que solo siente el que sabe que algo está mal, pero no sabe exactamente qué.

Fue ahí cuando abrí mis archivos en serio. Abajo describo, con brutal honestidad, lo que encontré.

El origen del caos: componentes "todoterrenos" controladores con obesidad mórbida

Si alguien hubiera revisado mis controladores al inicio, probablemente me habría prohibido seguir programando sin supervisión. Por ejemplo, tomemos el caso real de mi `CitationController.cs`:

- Tenía más de 300 líneas.
- Validaba permisos.
- Validaba datos.
- Tenía consultas directas a EF Core.
- Convertía entidades a DTOs... ¡pero a veces no!
- Controlaba estados de citas.
- Incluso hacía cálculos del intervalo entre dos fechas.

Era como un restaurante en el que el mesero toma la orden, cocina la comida, lava los platos, atiende la caja y abre la puerta.

No es sostenible.

No es mantenible.

No es testable.

No es profesional.

Lo peor es que yo mismo decía: "Pues funciona..."

Y eso es justamente lo más peligroso:

El código que "funciona" es el enemigo número uno de la ingeniería.

El anti-patrón que más me afectó: La lógica de negocio dispersa

A medida que avanzaba, la situación empeoraba. Descubrí algo que me dolió aceptar:

Mi lógica de negocio estaba regada por todo el backend.

Algunos ejemplos (reales):

- Un cálculo de disponibilidad médica estaba en el controlador de Citas.

- Otro cálculo similar estaba en el servicio de Horarios.
- Una validación de permisos estaba en el constructor del servicio.
- Algunas reglas estaban en los DTOs, otras en las entidades.
- Y otras —vergonzosamente— estaban directamente en el frontend Angular.

Esto provocaba que:

- Modificar una regla causara efectos secundarios en tres lugares distintos.
- Las pruebas fueran imposibles: para testear un método necesitaba levantar toda

la API.

- El rendimiento se volviera impredecible: EF Core hacía consultas redundantes.
- Se violara completamente el principio de responsabilidad única.

Básicamente, estaba escribiendo código que no se podía auditar.

Y en un sistema de salud, eso es imperdonable.

La solución: Arquitectura limpia REAL, no la de tutoriales

Inspirado por los artículos, decidí reestructurar todo hacia un enfoque más cercano

a:

- Clean Architecture
- DDD (Domain-Driven Design) ligero
- Patrones de separación de responsabilidades

No a nivel extremo —porque mi proyecto no necesita microservicios— pero sí lo suficiente para ser sostenible.

Cambié radicalmente cómo organizo las capas:

Capa 1: Controllers → Solo orquestación (tontos)

Los controladores dejaron de tomar decisiones.

Ahora solo dicen:

- "llama al servicio tal con estos datos"
- "devuélveme una respuesta estándar"
- captura esta excepción y devuélvela bonita"

Nada más.

Así el controlador deja de ser el cerebro del sistema y pasa a ser solo un recepcionista.

Capa 2: Services → Lógica de negocio centralizada (inteligentes)

Aquí ocurrió la cirugía mayor.

Mis servicios ahora hacen:

- Validaciones de reglas de negocio
- Coordinación entre repositorios
- Aplicación de modelos
- Conversión hacia DTOs
- Manejo adecuado de excepciones
- Registro estructurado

Y lo más importante:

todas las decisiones críticas están en un solo lugar, haciendo posible la trazabilidad y las pruebas unitarias.

Capa 3: Repositories → EF Core aislado correctamente

Antes yo mezclaba EF Core en todos lados.

Ahora tengo repositorios organizados como:

ICitationRepository

IClientRepository

IMedicalScheduleRepository

Y versiones en SQL Server como:

CitationRepository : ICitationRepository

Esto garantiza:

- Cambiar la base de datos es posible en el futuro
- Las consultas están optimizadas
- Los controladores no saben nada de EF Core

- Los servicios no conocen detalles de persistencia

Capa 4: Domain → Entidades puras con significado real

Este fue el cambio más profundo.

Antes mis entidades eran estructuras planas que reflejaban columnas.

Ahora representan el lenguaje del negocio.

Ejemplo:

Client ya no es solo alguien con nombre/cedula.

Ahora incluye:

- Métodos para validar su propio estado
- Reglas de negocio encapsuladas
- Comportamientos propios del dominio

Inspirado por el artículo de DDD (Özkan, 2025), entendí que:

”Si una entidad no sabe nada de sí misma, no es una entidad; es una bolsa de datos.”

DTOs: de paquetes improvisados a estructuras estandarizadas

Cuando leí sobre FHIR (Blessing, 2023), quedé en shock.

Comparé mis DTOs con los recursos FHIR y descubrí:

- mis nombres no seguían ningún estándar
- mis estructuras eran inconsistentes
- algunos DTOs tenían datos que no deberían viajar
- otros ocultaban información necesaria
- algunos endpoints devolvían entidades completas (¡grave error!)

Así que reescribí mis DTOs con tres objetivos:

1. Consistencia

Todos los endpoints devuelven respuestas unificadas:

```
{  
  
  success: true,
```

```
message: "OK",  
data: { ... }  
}
```

2. Seguridad

Nunca vuelvo a exponer IDs internos.

Nunca envío más datos de los necesarios.

3. Interoperabilidad

Mi ClientDto, PersonDto y CitationDto siguen ahora naming conventions compatibles con:

- FHIR Patient
- FHIR Appointment
- FHIR Practitioner

Aunque no implemento FHIR completo, mi backend ya habla un idioma parecido.

Esto lo aprendí de un principio simple:

”Tu sistema no debe ser una isla.”

Decisiones arquitectónicas difíciles: Monolito vs Microservicios

En medio de todo esto me dio por pensar:

”¿Y si convierto esto en microservicios?”

La moda del momento.

La trampa mortal del desarrollador novato.

Pero leyendo artículos de Hushare, 2024 y el estudio de Kbus Berru et al., 2020, comprendí algo clave:

Los microservicios no son mejores; son más complejos.

Solo valen la pena si tienes un equipo gigante y un sistema enorme.

Así que tomé la decisión madura:

→ Mantener un monolito modular, pero bien diseñado.

Esto me dio ventajas inmediatas:

- Menos latencia
- Menos puntos de fallo
- Más fácil de desplegar
- Docker Compose más simple
- Menos gastos computacionales
- Mejor rendimiento para salud pública

No necesito Kubernetes para pedir una cita médica.

El renacer del backend: pruebas, trazabilidad y orden

Después del reordenamiento arquitectónico, las cosas mágicamente empezaron a tener sentido:

- Mis errores dejaron de ser aleatorios

Porque ahora cada capa sabía exactamente qué hacer.

- Las pruebas unitarias se volvieron posibles

Un servicio pequeño, con responsabilidad definida, es testeable.

- EF Core dejó de hacer consultas duplicadas

Porque el repositorio controla todo.

- Modificar una regla no rompe todo el sistema

Porque está centralizada.

- El rendimiento mejoró sin cambiar una sola línea de SQL

Porque la arquitectura era más limpia.

- Entendí mi propio código

Y créeme: ese es el mejor regalo que un desarrollador puede darse.

Interoperabilidad: El Drama de Hablar Idiomas Diferentes

(Versión extendida, integrada con tu backend real y con referencias implícitas a FHIR y estándares médicos)

Durante meses pensé que la interoperabilidad era un lujo, una palabra elegante que solo usaban los hospitales grandes o los sistemas internacionales. Para mí, mientras mi

backend hablara con mi frontend, ya estaba bien. Pero cuando empecé a leer sobre FHIR y los problemas globales de los sistemas de salud, entendí que la interoperabilidad no es un lujo: es la base para que los datos de salud no se conviertan en un caos peligroso.

Y cuando comparé mis entidades con las estructuras del estándar FHIR, sentí ese mismo frío en la espalda que sentí cuando descubrí el desastre de mi arquitectura.

Era evidente: mi sistema hablaba un idioma que nadie más en el mundo podía entender.

Mi base de datos era una isla aislada en el océano

Cuando comencé a revisar mi base de datos (la de SQL Server que tenía montada en Docker), noté algo que antes ignoraba por completo: el nombre de mis tablas y campos no tenía ningún sentido semántico.

- Tenía una tabla llamada `tbl_clientes`
- Otra llamada `regCita`
- Otra llamada `tipoDocTbl`
- Una columna llamada `f_cita`
- Otra llamada `id_user`
- Y algunas joyas como `statusCitaEnumX3` (que ni yo entendía por qué tenía una "X3")

En mi cabeza, todo eso estaba "bien", porque así lo había nombrado yo.

Pero desde afuera, mi sistema era completamente incompatible con cualquier sistema médico real.

Ahí fue donde el artículo de Blessing et al., 2023 sobre FHIR me sacudió por dentro.

El problema no es técnico: es semántico

En el artículo explicaban algo que me partió la cabeza:

La interoperabilidad no falla por la tecnología; falla por el significado de los datos.

Puedes tener APIs REST, JSON, HTTPS, Docker, Kubernetes, GraphQL...

Pero si un hospital manda:

"patientId": 123

y otro manda:

clientIdentifier": 123

ya no están hablando del mismo concepto, aunque el número sea el mismo.

El problema no está en el código.

El problema está en el lenguaje.

Y ahí fue donde vi la gravedad en mi propio sistema.

Por ejemplo:

- Yo llamaba Client.^a lo que FHIR llama "Patient".
- Yo llamaba Üser.^a lo que FHIR usa como "Practitioner.^o RelatedPerson".
- A lo que yo llamaba Cita", ellos lo llaman .Appointment".
- Mi campo fecha_cita no tenía zona horaria.
- Mi campo status no seguía ningún estándar médico internacional.
- Y mis DTOs mezclaban conceptos: devolvían paciente + cita + documento +

médico, todo junto.

Era un desorden semántico completo.

Y un sistema de salud no puede permitirse ese lujo.

La revelación incómoda: Mi API tenía «dependencias culturales»

Había algo peor:

Mi sistema dependía del contexto cultural colombiano.

Ejemplos reales:

- Usaba Cédula como tipo de documento sin opción internacional.
- Mis direcciones eran asumidas como colombianas.
- Mis estados de cita eran "Pendiente / Completada / Cancelada", sin códigos

universales.

• Mis nombres seguían la estructura "Nombre / Apellido", no compatible con sistemas globales.

- Tenía campos llamados `.EPS` sin ningún equivalente internacional.

Eso significa que mi API no se podía usar fuera del país, y eso mata cualquier posibilidad de crecimiento real.

El artículo de FHIR es claro:

Para ser interoperable, tu sistema debe describir los datos de forma neutral, independiente del país, del hospital y del contexto.

La cruda realidad:

Mi sistema era localista, estrecho y completamente aislado.

El paso al cambio: adoptando FHIR sin reescribir el mundo

Obviamente yo no podía borrar toda mi base de datos ni rehacer mis entidades desde cero.

Pero sí podía hacer algo muy poderoso:

Adaptar mis DTOs para que hablaran un idioma estándar, aunque mis tablas internas siguieran siendo "imperfectas".

Esto fue un antes y un después.

Mis DTOs comenzaron a tener estructuras como:

Ejemplo de `PatientResponseDto` inspirado en FHIR:

```
{
  "identifier": {
    "type": "CC",
    "value": "1032456789"
  },
  "name": {
    "given": "Juan David",
    "family": "Artunduaga"
  },
  "birthDate": "1999-04-20",
```

```
"telecom": [  
  { "system": "phone", "value": "3214567890" },  
  { "system": "email", "value": "correo@example.com" }  
]  
}
```

Ejemplo de AppointmentResponseDto inspirado en FHIR:

```
{  
  "id": "APT-001",  
  "patient": "Juan David Artunduaga",  
  "practitioner": "Dr. Mauricio Noscue",  
  "status": "booked",  
  "start": "2025-01-10T14:30:00Z",  
  "end": "2025-01-10T15:00:00Z",  
  "location": "Consultorio 12 - Piso 2"  
}
```

Notarás algo:

Mis DTOs ahora tienen semántica internacional, incluso si las tablas internas todavía dicen `tbl_clientes`.

Ese fue el cambio más importante.

El patrón del "Traductor": la clave para no romper mi backend

Lo resolví así:

En mis servicios, antes de enviar datos al frontend, hago un proceso de:

Entidad interna → DTO compatible con FHIR → Frontend

Ejemplo real:

```
public PatientResponseDto ToPatientDto(Client client)
```

```
{  
    return new PatientResponseDto  
    {  
        Identifier = new IdentifierDto  
        {  
            Type = client.DocumentType,  
            Value = client.DocumentNumber  
        },  
        Name = new NameDto  
        {  
            Given = client.FirstName,  
            Family = client.LastName  
        },  
        BirthDate = client.DateBorn,  
        Telecom = new List<TelecomDto>  
        {  
            new TelecomDto("phone", client.PhoneNumber),  
            new TelecomDto("email", client.Email)  
        }  
    };  
}
```

Este patrón "traductor" se volvió esencial.

Me permitió modernizar el sistema sin destruirlo.

Un ejemplo real: el caos del "estado" de las citas

Antes yo tenía estados como:

- 0 = Pendiente
- 1 = Completada

- 2 = Cancelada
- 3 = No Asistió

Estos valores eran entendibles... pero solo para mí.

FHIR utiliza estados como:

- proposed
- pending
- booked
- arrived
- fulfilled
- cancelled
- noshow

Así que hice una conversión interna:

0 → pending

1 → fulfilled

2 → cancelled

3 → noshow

Ahora mi sistema deja de ser una isla semántica.

¿Por qué me importó tanto la interoperabilidad?

Porque no estoy construyendo una app de salud para mostrar pantallas bonitas.

Estoy construyendo algo que, en visión futura, podría integrarse con:

- un laboratorio clínico,
- un sistema de urgencias,
- una teleconsulta,
- un sistema de historia clínica electrónica,
- un reloj inteligente,
- una clínica de otro municipio,
- un registro nacional de salud.

Ninguno de esos sistemas va a aceptar JSONs con campos llamados `f_cita`.

La interoperabilidad no es un lujo.

Es la diferencia entre:

Sin interoperabilidad

→ Cada sistema es un mundo cerrado, aislado, inútil para otros.

Con interoperabilidad

→ Tu sistema se convierte en parte del ecosistema de salud.

Usabilidad: Diseñando para mi Abuela, No para Mí

(Versión extendida, profunda, con integración real a tu frontend Angular y móvil)

Cuando uno es joven y desarrolla software, cae en una trampa silenciosa: creer que los usuarios piensan, ven y entienden igual que uno.

Yo mismo caí en ella.

Mi interfaz en Angular se veía moderna, limpia, minimalista. Tenía íconos del Material Design, textos sobrios, menús elegantes. Para mí era "bonita", pero no estaba diseñada para quienes realmente la iban a usar.

Todo comenzó a cambiar cuando leí el artículo de Khamaj y Ali, 2024 sobre las dificultades que enfrentan los adultos mayores al utilizar aplicaciones móviles de salud.

Ese estudio me abrió los ojos de una forma que ningún tutorial de UI/UX lo habría hecho.

El error original: diseñé la aplicación pensando en mí

Voy a decirlo sin rodeos:

Diseñé la interfaz como si yo fuera el usuario promedio.

- Letra pequeña porque "se ve más elegante".
- Colores grises suaves porque son "premium".
- Menús escondidos detrás de iconos minimalistas porque "así está de moda".
- Formularios largos porque "es necesario pedir toda la información".
- Botones pequeños porque "los dispositivos ya tienen pantallas buenas".

Ese diseño funcionaba para mí.

Funcionaba para mis amigos.

Funcionaba para cualquier colega de tecnología.

Pero no funcionaba para:

- mi abuela,
- una persona con mala visión,
- un paciente con Parkinson,
- una persona mayor sin experiencia digital,
- alguien en una situación de urgencia real.

Ahí entendí el error.

Mi aplicación no estaba diseñada para las personas que más la necesitaban.

Primera prueba: imaginar a mi abuela usando la app

Hice un ejercicio sencillo.

Le presté mi teléfono a mi abuela y le pedí: ".Abue, intenta pedir una cita médica."

No pude olvidar su cara.

- Frunció los ojos para leer el texto.
- Tocó tres veces el mismo botón porque no reaccionaba visualmente.
- Se confundió al ver un icono de tres líneas (menú hamburguesa).
- No sabía si ".Especialidades." estaba dentro de "Servicios médicos." al revés.
- El formulario le pareció eterno.

Me preguntó: "¿Por qué todo es tan chiquito, mijito?"

Ese día me di cuenta de que no estaba haciendo una app de salud: Estaba haciendo una app para jóvenes sin problemas de visión.

Mi proyecto estaba fallando en su misión más importante: ser accesible, usable y humano.

La lectura que cambió todo: Usabilidad $\neq$ comodidad del programador

El estudio de Khamaj & Ali describía exactamente lo que yo estaba viviendo:

- Los menús complejos generan ansiedad en adultos mayores.
- Los textos pequeños son inaccesibles, aunque "se vean bonitos".
- La navegación poco clara aumenta el abandono.
- Los flujos largos cansan, frustran y confunden.
- La falta de retroalimentación visual genera inseguridad.

Y lo más fuerte:

„Un sistema usable no es el que funciona bien, sino el que no hace sentir torpe al usuario.”

Ahí se me partió el orgullo.

El rediseño: hacer la aplicación más humana

Comencé una refactorización completa del frontend, inspirada en cuatro pilares:

4.4.1 Pilar 1 — Tipografía adaptable (Dynamic Type)

Antes: Texto fijo en 12px, 14px o 16px.

Ahora: La app respeta el tamaño de letra que el usuario configuró en su celular.

- Si usa letra gigante → la app se vuelve gigante.
- Si usa letra mínima → la app se ajusta.

En Angular activé unidades relativas (rem) y eliminé tamaños fijos.

En React Native activé:

```
<Text allowFontScaling={true} adjustsFontSizeToFit={false}>
```

Esto cambió por completo la relación entre el usuario y la interfaz.

4.4.2 Pilar 2 — Contraste real, no «bonito»

Antes: Colores gris claro sobre gris oscuro (muy Dribbble, muy inútil).

Ahora: Colores con contraste alto, cumpliendo WCAG:

- Texto negro sobre fondo blanco
- Botones con colores sólidos
- Íconos gruesos

- Indicadores visuales claros

Descubrí algo sorprendente: Mientras más accesible, más profesional se ve.

4.4.3 Pilar 3 — Botones grandes, claros y separados

Antes: Botones minimalistas y pequeños.

Ahora: Superficies táctiles de mínimo 48dp, según Google Material.

Ejemplos reales del cambio:

Antes: [Agendar]

Después:

AGENDAR CITA

Mis métricas mostraron:

- Menos errores de toque
- Mayor sensación de seguridad
- Mayor velocidad en la navegación

4.4.4 Pilar 4 — Navegación simple y lineal

El usuario no debería pensar "Dónde estoy?" nunca.

Cambié la interfaz para:

- Reducir pantallas innecesarias
- Combinar pasos
- Que todo sea visible desde la pantalla principal
- Que los iconos tengan etiquetas
- Que el flujo de agendar una cita se pueda resumir en 3 toques

Ahora el flujo es:

1. Seleccionar especialidad
2. Seleccionar médico
3. Seleccionar horario
4. Confirmar

Nada más.

Nada menos.

Retroalimentación emocional: el detalle invisible

Hay algo que nunca consideré:

Las personas mayores se sienten inseguras usando tecnología.

Los botones no solo deben estar ahí.

Deben "sentirse vivos".

Así que agregué:

- Efectos de hover claros
- Botones que cambian de color al tocar
- Confirmaciones visibles
- Animaciones suaves al cargar
- Íconos que indican progreso

Los estudios muestran que esto reduce la ansiedad digital.

Y lo pude comprobar con pruebas reales.

Transparencia algorítmica: el usuario quiere saber por qué

Otro error que cometí:

Antes, si el sistema recomendaba un médico, lo hacía en silencio.

Eso genera desconfianza.

Siguiendo estudios sobre confianza algorítmica, implementé algo esencial:

Explicar el motivo detrás de cada decisión.

Ahora, cuando el sistema sugiere un doctor, el usuario lee:

- "Se te recomienda este médico porque está más cerca."
- "Se te asignó este horario porque es el más pronto disponible."
- "Tu cita fue reagendada porque el profesional ajustó su agenda."

Esto genera una sensación de justicia y control.

El rediseño en acción: pruebas reales con personas

Hice la prueba de oro:

Volví a poner la app en manos de mi abuela.

Ahora sí:

- leyó sin esfuerzo
- tocó donde debía
- entendió cada paso
- llegó al final sin confundirse

Me dijo algo que jamás voy a olvidar:

“Ahora sí entiendo cómo funciona. Antes me daba miedo dañarla.”

Ahí comprendí que la usabilidad no es decoración digital.

Es respeto por el usuario.

El aprendizaje más valioso

Lo que aprendí en esta etapa no está en ningún framework, ni en ningún repositorio de GitHub.

Lo aprendí observando, escuchando y aceptando que no soy el centro del universo.

Entendí que:

- La usabilidad es empatía.
- El diseño es responsabilidad.
- La accesibilidad no es opcional; es ética.
- Si una persona mayor no puede usar mi app, la app está mal.

Infraestructura: No Complicarse la Vida También es Ingeniería

(Versión ampliada, profunda, conectada con Appointments-Project-Back y con decisiones reales sobre Docker, monolitos, microservicios y sostenibilidad)

Cuando uno empieza a aprender sobre sistemas modernos, cae en la tentación de perseguir lo que “está de moda”.

Y en tecnología, la moda cambia cada tres meses.

En los últimos años, la religión del momento ha sido:

- Microservicios
- Kubernetes
- Nubes híbridas
- Multicloud
- Desacoplamiento extremo
- Orquestación distribuida
- Autoscaling inteligente

Y aunque esas tecnologías son increíbles, también son un arma de doble filo.

Especialmente para un estudiante... o para un proyecto que no necesita tanta maquinaria pesada.

Lo aprendí tarde, pero a tiempo.

El error de querer usar todo lo "profesional" sin necesidad real

Cuando levanté por primera vez mi sistema en Docker, sentí que era el rey del mundo.

Tener esto en mi docker-compose.yml me hacía sentir "arquitecto":

```
services:
```

```
  api:
```

```
    build: .
```

```
    ports:
```

```
      - "5149:8080"
```

```
    depends_on:
```

```
      - sqlserver
```

```
sqlserver:
```

```
  image: mcr.microsoft.com/mssql/server:2022-latest
```

```
  environment:
```

```
SA_PASSWORD: "Admin123!"
```

```
ACCEPT_EULA: "Y"
```

```
ports:
```

```
- "1437:1433"
```

Todo funcionaba.

Era portable.

Era profesional.

Pero luego cometí el error de pensar:

”Si ya uso Docker... ¿por qué no pasarme a microservicios?

¿Por qué no dividir mi API en diez partes: citas, usuarios, horarios, notificaciones, procedimientos, autenticación...?”

Empecé a escribir diagramas hexagonales complicadísimos.

Leí artículos sobre Kubernetes.

Me vi charlas completas de Netflix Engineering.

Hasta que leí los estudios de Hushare, 2024 sobre la complejidad en redes con microservicios, y el caso real de migración de Kbus Berru et al., 2020.

Y entendí algo fundamental:

Los microservicios no son más simples ni más profesionales.

Son más complejos, más frágiles y más caros.

Y lo peor de todo:

Un sistema pequeño en microservicios es peor que un monolito bien hecho.

Ahí frené en seco.

Mi sistema NO necesitaba microservicios (y admitirlo fue madurez técnica)

Me costó admitirlo porque el ego quiere escribir ”proyectos grandes”.

Pero un proyecto de citas médicas para un municipio NO requiere:

- diez despliegues independientes,
- un cluster Kubernetes,

- un mesh de servicios,
- logs distribuidos,
- tracing con Jaeger,
- circuit breakers,
- colas de mensajes internas,
- gateways avanzados.

Es más:

Agregar microservicios sin necesidad real solo añade puntos de fallo.

Los artículos lo repiten una y otra vez:

- Más servicios → más latencia.
- Más servicios → más fallas de red.
- Más servicios → más coordinación.
- Más servicios → más gasto.
- Más servicios → más dolores de cabeza.

Entendí que si intentaba meter microservicios en mi proyecto, iba a terminar dedicando más tiempo a configuraciones y devops que a resolver problemas reales de salud.

El monolito modular: la decisión inteligente para este proyecto

Así que tomé una decisión técnica sólida:

Mantener un monolito modular, pero modular, bien organizado y con fronteras internas c

Esto significa:

- Todo en una misma API .NET
- Una base de datos SQL Server
- Servicios internos bien separados
- DTOs y capas limpias
- Arquitectura escalable sin dividir redes
- Búsqueda de rendimiento dentro de un solo proceso

¿Los beneficios?

Beneficio 1: Rendimiento superior

Un monolito tiene:

- llamadas internas en memoria (rápidas),
- cero latencia de red entre módulos,
- menos overhead,
- menor consumo de CPU.

En salud, donde cada milisegundo importa, un monolito ligero es perfecto.

Beneficio 2: Menor complejidad operativa

Un solo contenedor con:

- API
- Servicios
- Modelo
- Autenticación
- Validaciones
- Controladores

es mucho más fácil de desplegar en:

- Windows
- Linux
- una máquina del hospital
- un servidor municipal
- una nube pequeña

Esto permite que el sistema pueda ser adoptado sin tener un ingeniero DevOps dedicado.

Beneficio 3: Seguridad centralizada

Con un solo backend puedo:

- controlar el acceso desde un mismo punto
- aplicar Zero Trust con un middleware
- tener logs unificados
- guardar auditorías en un único lugar

Si usuario quiere acceder a una cita, el flujo pasa por el mismo pipeline.

Esto es oro puro para un sistema de salud.

Beneficio 4: Costos bajos de mantenimiento

Un monolito:

- se actualiza rápido,
- se testea rápido,
- se despliega rápido,
- consume menos memoria,
- se monitorea más fácil.

En un municipio, esto es clave.

No siempre habrá presupuestos para super infraestructuras.

Inspiración académica: la decisión basada en evidencia

Los artículos que leí reforzaron mi decisión con argumentos sólidos:

Hushare, 2024:

Los microservicios incrementan la complejidad de diagnóstico, logging, tracing, seguridad, monitoreo.

Kbus Case Study (Berru, 2020):

La migración solo fue exitosa porque usaron un equipo completo, pruebas masivas y un rediseño total.

SDN Vulnerabilities (Haggag, 2021):

La distribución aumenta la superficie de ataque.

Multicloud Challenges (Baladari, 2024):

Cuanto más proveedores uses, más riesgos y más inconsistencias debes manejar.

Todo eso me explicó algo importante:

La madurez técnica no consiste en usar lo más moderno.

Consiste en elegir lo necesario, en cada contexto.

Sostenibilidad: donde la infraestructura se encuentra con la ética

Hay un aspecto de la infraestructura que casi nadie enseña:

el impacto ambiental.

Mi backend antes hacía esto:

- consultas repetidas,
- loops innecesarios,
- polling cada segundo desde el frontend,
- endpoints sin cache,
- procesos que nunca dormían.

Eso significa:

- más CPU
- más electricidad
- más costo
- más huella de carbono

Los artículos de optimización energética me mostraron algo que no esperaba:

El software también contamina.

Así que me propuse optimizar el consumo del sistema:

5.6 Optimización real: WebSockets sobre polling

Antes tenía esto en Angular/React Native:

```
setInterval(() => {  
  fetch('/api/notifications')  
}, 1000)
```

Miles de usuarios hacían esto → miles de llamadas por minuto.

Ahora uso SignalR en .NET:

```
await _hub.Clients.User(userId).SendAsync("NotificationReceived", data);
```

El servidor solo envía información cuando hay algo nuevo.

Resultado:

- Menos tráfico
- Menos CPU
- Menos batería en el teléfono
- Más velocidad real
- Menos emisiones

Infraestructura final del proyecto

El sistema quedó así:

Simple, robusto, profesional.

Sin adornos innecesarios.

La conclusión más importante de esta parte

La ingeniería no es presumir.

La ingeniería es elegir.

Y elegir lo correcto para el contexto es más profesional que querer sonar "avanzado".

Mi infraestructura no es un castillo de cristal.

Es una casa firme: sólida, directa, eficiente y sostenible.

Seguridad: Paranoia Saludable para Proteger Datos Médicos

(Versión ampliada — profunda, técnica, humana y completamente integrada a tu backend y tus artículos)

Cuando inicié el proyecto, pensaba que la seguridad se resolvía con estas dos líneas mágicas:

```
[Authorize]
```

```
public class CitationController : ControllerBase
{
}
```

Creía que con eso ya nadie podía entrar.

Creía que, porque mi API estaba en localhost dentro de Docker, estaba protegida.

Creía que los ataques eran algo que les pasa a las empresas grandes, no a un estudiante.

Luego entendí la verdad más incómoda de todas:

Si manejas datos de salud, eres un objetivo.

Si manejas información personal, eres un objetivo.

Si conectas un sistema a internet, eres un objetivo.

Incluso si nadie quiere atacarte, una mala configuración puede destruirlo todo.

Y cuando leí el artículo sobre Zero Trust, las vulnerabilidades de SDN, la inseguridad de las PYMEs colombianas y la necesidad de auditorías algorítmicas, comprendí que mi sistema estaba desnudo.

Completamente expuesto.

Inseguro desde las entrañas.

Y eso me dio miedo.

El primer golpe: mi API confiaba demasiado

El paradigma viejo dice:

”Si la petición viene de tu propia web, confía.

Si viene de dentro de tu red, confía.

Si el usuario ya inició sesión una vez, confía.”

Pero el artículo de Baladari, 2024 sobre Zero Trust es claro:

No confíes en NADA. No confíes en NADIE.

Todo se valida.

Todo se verifica.

Todo se registra.

Y cuando revisé mi propio backend, encontré fallas graves:

- No validaba el origen de las peticiones

Podían enviarme un POST desde Postman y yo lo aceptaba como si fuera la app oficial.

- No verificaba el contexto del usuario Un usuario sin rol de doctor podía acceder a endpoints internos si sabía la URL.

- Tenía endpoints que devolvían mucha información

Algunos devolvían incluso objetos completos de EF Core, con propiedades que no deberían salir del servidor.

- No tenía auditoría interna

No sabía quién había hecho qué, ni cuándo.

- No tenía un sistema de tokens robusto

Mi JWT no incluía claims suficientes para controlar permisos reales.

- Permitía IDs directos

Un atacante podía intentar cambiar su propia cita adivinando ID 102, 103, 104...

Me di cuenta de que estaba jugando con fuego.

6.2 Implementando Zero Trust en serio

Cambié completamente mi mentalidad.

Zero Trust no es un framework ni una librería. Es una filosofía completa.

La regla es simple:

Nunca asumas que una petición es confiable.

Obliga a que te lo demuestre.

Cada. Vez.

Así que adopté las siguientes medidas:

Autenticación estricta con tokens cortos

Antes mis tokens duraban horas.

Ahora duran minutos, como recomiendan los artículos de seguridad.

- Implementé renovación silenciosa desde el frontend
- Expiración rápida en el backend
- Tokens más pequeños, limpios y precisos

Ejemplo:

```
var token = new JwtSecurityToken(  
    issuer: "...",  
    audience: "...",  
    claims: userClaims,  
    expires: DateTime.UtcNow.AddMinutes(20)  
);
```

Claims detallados, no solo roles

Antes tenía un JWT con:

role: "user"

Ahora tengo claims como:

role: "patient"

permissions: ["cite.read", "cite.create"]

tenancy: "paciente:1001"

medical_authority: false

Estos claims permiten:

- restringir cada endpoint
- definir permisos a nivel granular
- asociar un paciente solo a SU información
- evitar que adivinen IDs
- mantener trazabilidad

Esto lo aprendí del artículo de Dowding y Taylor, 2024:

La confianza algorítmica nace de la transparencia y la capacidad de auditar.

Control de acceso por "contexto" y no por rol

Roles son insuficientes.

Un paciente y un médico pueden tener el mismo rol si ocurre un error.

Así que implementé verificaciones como:

- Verificación de identidad cruzada

Si la ruta es:

GET /api/citation/patient/2003

El backend compara:

patientId == claim.tenancy

Si no coincide → 403.

- Verificación de estado del usuario

Un usuario suspendido no puede hacer nada, aunque tenga token válido.

- Verificación horaria

Solo personal médico autorizado puede modificar agendas en ciertos tramos.

Auditoría: saber quién hizo qué, cuándo y desde dónde

Uno de los artículos (Sánchez-Sánchez, 2021) mostraba que la mayoría de fallos en PYMEs no es culpa de hackers, sino de:

- errores humanos,
- configuraciones débiles,
- versiones desactualizadas,
- permisos otorgados de más,
- falta de monitoreo.

Ahí comprendí que mi backend no registraba nada.

Así que creé una tabla de auditoría:

AuditLog

Id

UserId

Action

Endpoint

PayloadBefore

PayloadAfter

Timestamp

IpAddress

Device

Ahora puedo:

- reconstruir una acción completa
- ver si alguien manipuló datos de forma indebida
- detectar patrones extraños
- cumplir con normas básicas de salud

Esto le dio mucha más confianza a mi sistema.

Cifrado: que un atacante no entienda nada aunque robe todo

Otro error común:

Pensar que porque la base de datos está en Docker → está segura.

Falso.

Un atacante con acceso a la máquina puede sacar el volumen completo.

Así que tomé estas medidas:

- Cifrado de contraseñas con hashing seguro

Nada de MD5, SHA1 o cosas viejas.

Uso:

PasswordHasher<User>

con salting y hashing múltiple.

- Cifrado de campos sensibles

No todo se cifra, porque afectaría performance, pero sí:

- teléfonos
- correos electrónicos
- tokens internos
- HTTPS en toda la comunicación

Incluso local (lo recomienda Zero Trust).

SDN y la verdadera complejidad de la red

El artículo de Haggag y Hanafy, 2021 dice algo muy fuerte:

”Las redes definidas por software centralizan el control, lo que crea un punto único de ataque.”

Esa frase me hizo replantear mis decisiones.

Si algún día escalo este proyecto a un hospital real, o a varias clínicas municipales, podría caer en la tentación de usar:

- SDN,
- microservicios,
- gateways inteligentes.

Pero los estudios lo demuestran:

Entre más distribuido, más inseguro si no tienes un ejército de profesionales.

Por eso:

- Mantener el monolito te simplifica la vida
- Centraliza la seguridad
- Reduce la superficie de ataque
- Minimiza el riesgo de DDoS interno
- Facilita la auditoría

Pruebas automatizadas de seguridad: IA a mi favor

El artículo de IA para pruebas visuales (Ivanova, 2020) me inspiró a pensar:

¿Y si mis formularios visuales presentan errores que dejan expuestos datos de salud?

Así que consideré:

- pruebas automáticas de campos sensibles
- validación visual del DOM
- detección de fugas de información
- análisis de patrones anómalos

No implementé todo, pero sí dejé preparado:

- estructura para tests
- mocks seguros
- sanitización del DOM
- restricciones de visualización

Un sistema de salud NO puede permitir ambigüedad

Aprendí esto:

En comercio electrónico → un error cuesta dinero

En redes sociales → un error molesta

En salud → un error pone a una persona en riesgo

Mis endpoints ahora:

- validan
- autentican
- verifican claims
- auditan
- filtran datos
- limitan acceso
- registran anomalías

El aprendizaje emocional detrás de la seguridad

La seguridad me enseñó a:

- desconfiar como un paranoico,
- ser humilde con el código,
- no dar nada por hecho,

- diseñar pensando en lo peor,
- proteger incluso al usuario que no sabe que está en riesgo,
- construir sistemas donde los errores no destruyan vidas.

Nunca voy a olvidar esta frase que escribí una noche:

”Si diseño mal la seguridad, no fallo yo.

Falla alguien que confió en mí.”

Las Tecnologías que Decidí NO Usar (Y Por Qué Eso Es Bueno)

(Versión ampliada, reflexiva, coherente con los artículos de AR, IA, RISC-V, y pruebas visuales)

Muchas veces en los proyectos de software se valora más lo que se incluye que lo que se decide dejar por fuera.

Pero, con el tiempo, entendí que un buen ingeniero no es el que mete más cosas... sino aquel que sabe qué dejar afuera para que el sistema no se vuelva un monstruo incontrolable.

Durante la lectura de los artículos, hubo muchas tecnologías que me emocionaron. Me imaginé usando de todo: • Realidad aumentada • Sensores biométricos • Procesadores de bajo consumo • IA para testing • Blockchain • Sistemas de edge computing • Redes definidas por software • Microservicios predictivos con IA • Monitorización inteligente

Pero aunque esas tecnologías son increíbles, también son un arma de doble filo. Especialmente para un estudiante... o para un proyecto que no necesita tanta maquinaria pesada.

Lo aprendí tarde, pero a tiempo.

Realidad aumentada: tecnología increíble, momento incorrecto

Uno de los artículos más fascinantes que leí fue el trabajo de Sahu et al., 2025 sobre optimización energética para aplicaciones de realidad aumentada usando edge computing.

Me imaginé a los pacientes: • viendo con su cámara dónde está el consultorio, • explorando rutas en 3D, • recibiendo instrucciones visuales para llegar a su cita.

La idea era espectacular.

Pero luego pensé en la abuela que probó mi app. Ella apenas podía leer un botón. ¿Para qué meter realidad aumentada si: • consume batería, • requiere teléfonos potentes, • distrae del objetivo real, • aumenta la complejidad técnica, • y no genera verdadero valor en este contexto?

Apliqué la madurez que no tenía antes: No todo lo que es genial es necesario.

Aun así, algo valioso sí tomé del artículo: Aprendizaje técnico del artículo: • Las aplicaciones deben delegar tareas pesadas a servidores cuando sea posible. • El edge computing puede reducir consumo en dispositivos móviles. • La adaptación dinámica de calidad mejora la estabilidad.

Y eso SÍ lo implementé en mi backend con: • WebSockets • reducción de peticiones repetidas • procesos más ligeros • lógica solo cuando es necesaria

Wearables y procesadores de ultra bajo consumo

También me atrapó el artículo del procesador RISC-V Day-Night, capaz de monitorear signos vitales consumiendo casi nada de energía.

Y pensé: "Qué increíble sería que mi sistema recibiera alertas del reloj de un paciente."

Pero la realidad técnica es otra: • no tengo dispositivos médicos homologados, • no tengo APIs de wearables, • no tengo sensores certificados, • y no tengo el hardware necesario.

Aun así, reorganicé mi base de datos para preparar "el futuro": Agregué estructuras como: • AlertType • MeasurementSource • SyncTimestamp • DeviceIdentifier

De modo que si algún día conecto wearables, el sistema ya está listo.

Eso es ingeniería: no meter tecnología que no aplica HOY, pero dejar listo el camino para MAÑANA.

Inteligencia artificial para pruebas visuales: poderosa, pero innecesaria ahora

El artículo de Ivanova et al., 2020 sobre pruebas visuales automatizadas con IA fue brutalmente convincente.

Te permite: • detectar errores de UI sin falsos positivos, • analizar intenciones y no solo píxeles, • automatizar validaciones, • reducir tiempo de QA.

Era de esos artículos que te hacen decir: "¡Wow, quiero esto ahora mismo!"

Pero entonces llegó la pregunta lógica: ¿En qué etapa está mi proyecto? Mi frontend, aunque funcional, todavía está cambiando cada semana. Los estilos, los colores, los botones, los textos... nada está completamente congelado.

Implementar IA para testing visual ahora habría sido como ponerle turbo a un carro sin terminar.

Además: • El costo computacional es alto. • Se requiere entrenamiento de modelos. • El tiempo de integración no compensa la etapa actual del proyecto. • El beneficio es más útil cuando la UI es estable.

Pero aunque no implementé IA para el testing visual, sí apliqué las ideas: • mejoré retroalimentación visual, • estandaricé componentes, • eliminé inconsistencias, • optimicé el DOM para pruebas futuras, • hice capturas automatizadas para comparar pantallas.

No tengo IA... pero tengo un proceso de testing más inteligente.

Blockchain: poderoso, pero demasiado grande para este momento

Uno de los artículos hablaba de confianza algorítmica (Dowding, 2024).

Otro de multicloud y protección de privacidad (Baladari, 2024).

Y otro mencionaba DLT (Hushare, 2024).

Me imaginé algo increíble: • historia clínica inmutable, • auditorías criptográficas, • transparencia total, • seguridad a nivel matemático.

Pero luego pensé: • ¿Necesito blockchain para agendar una cita en un centro médico local? • ¿Tengo la infraestructura para correr nodos? • ¿Quién validará las transacciones? • ¿Quién pagará el almacenamiento? • ¿Cómo se integra con sistemas reales?

La respuesta era evidente: • Interesante • Futuro posible • No necesario hoy

Así que guardé esa idea para la segunda versión del proyecto.

Microservicios con IA para escalabilidad: no en este proyecto

Otro tema que me emocionó fue: • microservicios predictivos • escalamiento automático • procesamiento distribuido inteligente • SDN programables

Todo eso sale de los artículos: • SDN y riesgos de DDoS (Haggag, 2021) • Multicloud inteligente (Baladari, 2024) • Edge Computing (Sahu, 2025) • Migración de microservicios (Berru, 2020)

Pero ahí entendí el mensaje principal: La complejidad que no necesitas es un problema, no una mejora.

El sistema de citas médicas requiere: • estabilidad • simplicidad • mantenimiento fácil • costo bajo • seguridad sólida

NO requiere 10 microservicios corriendo en paralelo con IA.

La clave: Elegir lo adecuado, no lo impresionante

Algo que me quedó claro al leer todos tus artículos fue: • La ingeniería no es una vitrina • La ingeniería es toma de decisiones • Las decisiones correctas se basan en contexto • El sistema debe ser útil, no extravagante

Y así, estas fueron mis decisiones: • No usar AR → innecesario • No usar IA para testing → inmaduro aún • No usar blockchain → demasiado grande • No usar microservicios → sobreingeniería • No usar SDN → más riesgos que beneficios

Sí usar: • arquitectura limpia • monolito modular • interoperabilidad con FHIR • accesibilidad real • seguridad Zero Trust • WebSockets optimizados • DTOs estandarizados • patrones mantenibles • pruebas manuales bien diseñadas • documentación profesional • señalización clara para el usuario

Esto hace que el sistema sea: • humano • sostenible • mantenible • compatible con estándares • éticamente responsable

El verdadero valor de lo que dejé por fuera

Podría resumirlo así: • Las tecnologías que NO usé Habrían aumentado la complejidad.

- Las tecnologías que SÍ usé Aumentan la calidad.

Me convertí en un desarrollador más maduro. Entendí que "más" no es "mejor". Y que incluso decir NO es una decisión técnica útil.

(Y por qué eso es bueno) – La Decisión de Renunciar también es Ingeniería

(Versión ampliada, reflexiva, coherente con los artículos de AR, IA, RISC-V, y pruebas visuales)

Hay un momento en todo proyecto tecnológico en el que uno tiene que tomar una decisión muy poco glamorosa, pero absolutamente crucial: descartar ideas.

Este capítulo es precisamente sobre eso. Porque si algo aprendí leyendo los artículos, revisando la teoría y trabajando directamente sobre mi proyecto Appointments-Project-Back, fue lo siguiente:

El software profesional no se mide por la cantidad de tecnología que usa, sino por la calidad de las decisiones que toma.

Y muchas veces, como en mi caso, la decisión correcta es no implementar algo, aunque suene espectacular, aunque esté de moda, aunque se vea impresionante en un diagrama o un video de YouTube.

Esta parte del documento es la más honesta, porque habla de renunciar. Renunciar a cosas "cool", renunciar a ideas ambiciosas, renunciar a herramientas de vanguardia... por el simple hecho de que no eran necesarias, no aportaban valor inmediato, o no estaban alineadas con las necesidades reales de los usuarios.

Renunciar a la Realidad Aumentada (AR): una fantasía hermosa pero impráctica

Uno de los artículos más fascinantes que leí fue el trabajo de Sahu et al., 2025 sobre optimización energética para aplicaciones de realidad aumentada usando edge computing.

Hablar de AR es emocionante: imaginar a los pacientes caminando por el hospital guiados por flechas virtuales, o viendo una representación 3D del consultorio, o incluso visualizar la ruta hasta el médico directamente desde el celular.

Pero cuando comparé esa idea con mi realidad, tuve que aceptarlo:

- La AR no aportaba nada esencial a un sistema de citas médicas.
- Consumía demasiada batería (algo crítico en salud).
- Aumentaba el tiempo de desarrollo.
- Aumentaba los requisitos del dispositivo móvil.
- Añadía complejidad sin mejorar la experiencia del flujo principal.

Esa fue una lección madura: No porque exista una tecnología debes usarla.

Debes usar aquello que mejora la vida del usuario, no tu ego como programador.

Aun así, algo valioso sí tomé del artículo: Aprendizaje técnico del artículo: • La importancia de delegar tareas pesadas al servidor. • El concepto de offloading para reducir consumo energético. • La idea de tomar decisiones según el estado del dispositivo.

Y eso sí terminé aplicado en mi proyecto a través de optimizaciones como: • WebSockets en lugar de polling. • Procesamiento de notificaciones desde SignalR. • Reducción de llamadas innecesarias a la base de datos.

A veces, renunciar a una idea te permite rescatar su esencia y aplicarla con sabiduría.

No implementar relojes inteligentes (por ahora): pensar a largo plazo

Otro artículo que me estremeció fue el de Choi et al., 2024 sobre procesadores RISC-V de consumo ultrabajo, diseñados para wearables capaces de detectar anomalías cardíacas.

La idea de que mi sistema pudiera recibir alertas desde un reloj inteligente en tiempo real —por ejemplo, un posible infarto— es increíble.

Sería un caso de uso digno de una startup de Silicon Valley.

Pero una cosa es soñar. Otra muy diferente es entregar un proyecto factible, estable

y mantenible.

Implementar soporte para wearables implicaría: • obtener dispositivos reales • integrar APIs de fabricantes (Apple, Samsung, Huami) • manejar BLE o APIs propietarias • integrar datos biométricos sensibles • procesarlos según regulaciones médicas • contar con hardware certificado (cosa que no tengo)

Y siendo realistas: • No tenía dispositivos. • No tenía presupuesto. • No tenía necesidad clínica inmediata. • No tenía las cadenas de seguridad necesarias para biometría. • No tenía validación médica para procesar señales fisiológicas.

Así que tomé la decisión madura: "El proyecto debe estar preparado para el futuro, pero no tiene que implementarlo desde el primer día."

Lo que sí dejé listo fue: • la estructura de base de datos para soportar "alertas externas", • un endpoint genérico para recibir notificaciones de dispositivos, • una arquitectura modular que permita integrar wearables más adelante, • documentación para una futura integración con IoT.

Mientras que el feature final no se implementó, la preparación para el futuro sí quedó.

Ese es el sello de un diseño inteligente.

No usar IA para Testing Visual (por ahora): priorizar el tiempo y los recursos

El artículo de Ivanova et al., 2020 sobre pruebas visuales automatizadas con IA fue uno de mis favoritos.

Mostraba cómo usar segmentación de imágenes con Mean Shift para detectar errores visuales sin tener que comparar capturas pixel a pixel.

Era de esos artículos que te hacen decir: "¡Wow, quiero esto ahora mismo!"

Pero entonces llegó la pregunta lógica: ¿En qué etapa está mi proyecto?

Mi frontend, aunque funcional, todavía está cambiando cada semana.

Los estilos, los colores, los botones, los textos... nada está completamente congelado.

Implementar IA para testing visual ahora habría sido como: • Instalar sensores

láser para medir la torre... cuando todavía estás moviendo los ladrillos.

Además: • El costo computacional es alto. • Se requiere entrenamiento de modelos.
• El tiempo de integración no compensa la etapa actual del proyecto. • El beneficio es más útil cuando la UI es estable.

Pero aunque no implementé IA para el testing visual, sí apliqué las ideas: • mejoré retroalimentación visual, • estandaricé componentes, • eliminé inconsistencias, • optimicé el DOM para pruebas futuras, • hice capturas automatizadas para comparar pantallas.

No tengo IA... pero tengo un proceso de testing más inteligente.

Renunciar para poder avanzar: una habilidad que no enseñan en los cursos

Este es el punto clave de esta sección.

Mientras estudiaba, me di cuenta de que ningún tutorial de YouTube, ningún curso básico de .NET, y ningún ejemplo de StackOverflow te enseña esto:

En ingeniería, no todo se trata de agregar.

Mucho se trata de elegir qué NO agregar.

Mis renunciaciones no fueron un fracaso.

Fueron decisiones de diseño.

Decisiones que: • me permitieron entregar un sistema estable, • evitaron complejidad innecesaria, • mantuvieron el proyecto dentro del alcance, • prepararon una base escalable, • enfocaron los esfuerzos donde realmente importaba.

Si hubiera intentado implementar: • AR, • IoT biomédico, • AI visual testing, • microservicios, • multicloud, • edge computing avanzado,

... habría terminado con un Frankenstein imposible de desplegar, mantener, probar o presentar.

El éxito técnico consiste en saber abandonar.

El resultado: un proyecto realista, sólido y preparado para el futuro

Al final de esta etapa, mi proyecto quedó estructurado así:

Lo que implementé • Arquitectura limpia • Seguridad Zero Trust • Performance

con WebSockets • Interoperabilidad semiestándar FHIR • Accesibilidad real • Backend robusto • Frontend usable para personas mayores • Infraestructura ligera (Docker)

Lo que preparé, pero NO implementé (todavía) • Integración para wearables • IA para test visual • Soporte nativo para AR • Edge computing para offloading pesado • Microservicios escalados

Lo que descarté completamente (por ahora) • Migración a Kubernetes • API para datos biomédicos complejos • Pruebas basadas en algoritmos de visión • Flujo de geolocalización con AR

El proyecto final es coherente con su propósito: gestionar citas médicas de forma segura, rápida, accesible y sostenible.

Todo lo demás, aunque tentador, habría sido una distracción.

La frase que resume toda esta sección

Si tuviera que resumir esta parte en una línea, sería esta:

Renunciar no es retroceder. Renunciar es diseñar.

Conclusiones: Dejando de Ser Amateur

(Versión ampliada, emocional, técnica y con introspección personal + relación con los 20 artículos)

Llegar hasta aquí ha sido mucho más que terminar un proyecto de grado. Ha sido un proceso de transformación personal, un choque frontal entre lo que yo creía que era "desarrollar software" y lo que realmente significa hacer ingeniería. Si algo ha quedado claro durante este viaje es que ninguna línea de código, por más elegante que parezca, vale la pena si no está sostenida por fundamentos, por experimentación, por teoría y, sobre todo, por responsabilidad humana.

Leer los 20 artículos no fue cumplir un requisito académico: fue una confrontación.

Una conversación incómoda entre mi yo antiguo —rápido, improvisado, confiado— y un nuevo yo más crítico, más paciente y, honestamente, más humilde.

Estas conclusiones no son un resumen técnico: son reflexiones de vida, aprendizaje

profesional, errores corregidos y decisiones maduras que cambiaron la forma en que entiendo la ingeniería de software.

La primera gran lección: "Si funciona" no es suficiente

Antes de este proceso, yo pensaba que cuando una API devolvía "200 OK", ya podía celebrar. Creía que si un formulario cargaba y no se rompía, era un éxito. Que si Docker levantaba mis contenedores sin explotar, ya había hecho un buen trabajo.

La teoría me obligó a reconocer que eso es apenas el mínimo vital.

Los artículos sobre arquitectura (AppCraft, Clean Architecture, DDD), sobre rendimiento, sobre microservicios mal aplicados, sobre seguridad Zero Trust, sobre confianza algorítmica... todos repetían un mensaje silencioso:

"La calidad no se mide por la ausencia de errores visibles, sino por la solidez invisible del sistema."

Mi backend funcionaba. Sí.

Pero no era sólido.

Tenía lógica dispersa, DTOs confusos, endpoints que devolvían cosas distintas aunque hicieran lo mismo, nomenclaturas incoherentes y decisiones diseñadas "al ojo".

Hoy, gracias a este proceso, descubrí que:

- La coherencia arquitectónica importa.
- Los estándares importan.
- La semántica importa.
- La accesibilidad importa.
- La seguridad importa.
- La trazabilidad importa.
- La interoperabilidad importa.

Y lo más doloroso de aceptar:

Funcionar $\neq$ estar bien hecho.

La segunda gran lección: la ingeniería es invisible

Lo que se ve en pantalla —los colores, los botones, los íconos— es apenas el 5 % de la historia.

El resto es invisible: decisiones, patrones, validaciones, modelos, reglas, servicios, repositorios, estándares, contratos, seguridad.

Los artículos sobre usabilidad para adultos mayores me mostraron que hasta un simple botón tiene profundidad:

- ¿Es legible?
- ¿Es alcanzable?
- ¿Es entendible?
- ¿Produce ansiedad?
- ¿Respeto el tamaño configurado del teléfono?

Codificar un botón es fácil.

Diseñar un botón respetando la dignidad de una persona mayor... eso es ingeniería.

Entendí que gran parte del trabajo profesional no es visible cuando todo funciona bien.

El mundo solo nota la ingeniería cuando falla.

Y ese es el punto:

mi proyecto ya no depende de la suerte.

La tercera gran lección: priorizar es más maduro que presumir

Yo quería usar:

- Kubernetes
- Microservicios
- Nubes híbridas
- Cajas de eventos
- Funciones serverless
- Bases distribuidas

- Mensajería entre servicios
- AI para testing visual

Porque se ve "profesional".

Porque suena "avanzado".

Porque se escucha impresionante cuando uno lo cuenta.

Pero los artículos sobre multicloud, SDN, migraciones fallidas y complejidad distribuida me dieron el golpe final:

La ingeniería no es usar lo más moderno.

La ingeniería es usar lo adecuado.

Y lo adecuado para mi proyecto era:

- un monolito modular
- una arquitectura limpia
- una base SQL Server estable
- un backend optimizado
- señalización en tiempo real con SignalR
- buena seguridad
- buena usabilidad
- DTOs compatibles con FHIR

Ese conjunto —simple pero sólido— es cien veces más valioso que veinte microservicios mal coordinados.

La madurez está en decir:

"No necesito eso... todavía."

La cuarta gran lección: la teoría sin práctica es estéril; la práctica sin teoría es peligrosa

Cada artículo tuvo impacto real en mi proyecto:

- AppCraft → me enseñó a separar estructura de presentación.
- Clean Architecture → reorganizó todo mi backend.

- FHIR → me obligó a corregir mis DTOs y pensar en interoperabilidad.
- Usabilidad → me hizo rediseñar la aplicación pensando en personas reales.
- Zero Trust → reescribió por completo mi seguridad.
- Kbus → me enseñó cuándo NO usar microservicios.
- Docker vs VM → confirmó mis decisiones de despliegue.
- Edge Computing → inspiró mis mejoras de eficiencia.
- AI en Testing → me mostró hacia dónde puedo evolucionar en el futuro.
- Toma de decisiones → me enseñó la importancia de la transparencia.

Cada texto académico tuvo una consecuencia directa, medible y visible en mi código.

Y a su vez, mi código mejorado me permitió entender mejor la teoría.

Ese ciclo —leer, aplicar, comparar, refinar— es lo que convierte el aprendizaje en experiencia.

La quinta gran lección: el software de salud no es un proyecto cualquiera

Este fue el aprendizaje más emocional de todos.

Un sistema de citas médicas no es un videojuego.

No es un e-commerce.

No es una red social escolar.

No es una página que "si falla, no pasa nada".

Aquí:

- un error puede hacer que alguien pierda una consulta,
- una mala UX puede hacer que una persona mayor no pueda pedir cita,
- un endpoint inseguro puede filtrar datos sensibles,
- una mala decisión técnica puede retrasar atención médica.

Los artículos sobre usabilidad, algoritmos, interoperabilidad y seguridad no solo reforzaron lo técnico:

reforzaron el compromiso ético.

Comprendí que cada decisión que tomé —nombre de una columna, tamaño de un botón, estructura de un JSON, manejo de un token— tiene impacto humano.

La sexta gran lección: documentar no es escribir—es entender

Antes pensaba que documentar era:

- copiar y pegar
- hacer un resumen
- escribir lo que ya hice

Pero este documento —estas 30–40 páginas— no son un requisito.

Son mi registro de evolución técnica.

Cada frase que escribí me obligó a:

- repensar decisiones,
- revisar código,
- cuestionar mi arquitectura,
- justificar mis elecciones,
- detectar inconsistencias,
- validar mis ideas,
- entender mis propios errores.

Documentar se convirtió en el espejo más honesto del proyecto.

La conclusión final: ahora sí soy ingeniero, no solo programador

Antes construía pantallas y endpoints.

Hoy construyo sistemas.

Antes seguía tutoriales.

Hoy tomo decisiones.

Antes programaba al instinto.

Hoy programo con fundamentos.

Antes mi proyecto funcionaba "de casualidad".

Hoy funciona porque está diseñado para funcionar.

Eso es lo que me llevo de este viaje académico y técnico:

La diferencia entre un programador y un ingeniero no es el lenguaje que usa ni las herramientas que domina.

La diferencia está en la intención, la responsabilidad y la coherencia detrás de cada línea de código.

Y si mañana alguien decide continuar mi proyecto, integrarlo, expandirlo o auditarlo, encontrará algo que antes no existía:

- un sistema entendible,
- un sistema profesional,
- un sistema responsable,
- un sistema digno de confianza.

Ese es el verdadero logro.

Esa es la verdadera calificación.

Referencias Bibliográficas

1. Alwakeel, L., et al. (2025). *AppCraft: Model-driven development framework*. IEEE Access.
2. Blessing, A., et al. (2023). *Introduction to FHIR*. ResearchGate.
3. Khamaj, A., & Ali, M. (2024). *Examining usability challenges for older adults*. Alexandria Eng. Journal.
4. Dowding, K., & Taylor, B. (2024). *Algorithmic decision-making and trust*. Philosophy & Technology.
5. Mulla, F. (2024). *Choosing the Best Architecture for Mobile Applications*. ResearchGate.
6. Alomari, A. H., et al. (2023). *Dual-phase resource allocation in SDN*. IEEE Access.

7. Haggag, A., & Hanafy, D. (2021). *Network Performance and Security of SDN*. ResearchGate.
8. Baladari, V. (2024). *Enhancing Security in Multi-Cloud Environments*. ResearchGate.
9. Sahu, D., et al. (2025). *Edge assisted energy optimization*. Scientific Reports.
10. Ivanova, K., et al. (2020). *AI in Automated Visual Testing*. CEUR Workshop.
11. Biørn-Hansen, A., et al. (2020). *Cross-platform vs Native Performance*. Empirical Software Engineering.
12. Özkan, O., et al. (2025). *Domain-Driven Design review*. Journal of Systems and Software.
13. Amarteyfio, P. K. (2023). *Ubuntu: An Ashesi group work platform*. Ashesi Repository.
14. Berru, Y. T., et al. (2020). *Migración de un monolito a microservicios*. Dialnet.
15. Sánchez-Sánchez, P. A., et al. (2021). *Seguridad informática en PYMEs*. Información tecnológica.
16. Hushare, J. V. (2024). *Interoperability framework*. ScienceDirect.
17. Potdar, A. M., et al. (2020). *Performance evaluation of docker*. Procedia Computer Science.
18. Choi, E., et al. (2024). *Day-Night architecture for wearables*.
19. Praveena, A. (2025). *Chronic Connect Mobile Health App*.

Referencias

- Baladari, V. (2024). Enhancing Performance and Security in Multi-Cloud and Hybrid-Cloud Environments. *ResearchGate*.
- Berru, Y. T., et al. (2020). Migración de un monolito a una arquitectura basada en microservicios. *Dialnet*.
- Blessing, A., Joy, M., & Brown, K. (2023). Introduction to FHIR (Fast Healthcare Interoperability Resources) and its Role in Health Data Interoperability. *ResearchGate*, 1-10.
- Choi, E., et al. (2024). Day-Night architecture: Development of an ultra-low power RISC-V processor [Referencia interna].
- Dowding, K., & Taylor, B. (2024). Algorithmic decision-making, agency costs, and institution-based trust. *Philosophy & Technology*, 1-10.
- Haggag, A., & Hanafy, D. (2021). Network Performance and Security Analysis of Software Defined Networking. *ResearchGate*.
- Hushare, J. V. (2024). Interoperability framework to enhance DLT based systems. *ScienceDirect*.
- Ivanova, K., et al. (2020). Artificial Intelligence in Automated System for Web-Interfaces Visual Testing. *CEUR Workshop Proceedings*.
- Khamaj, A., & Ali, M. (2024). Examining the usability and accessibility challenges in mobile health applications for older adults. *Alexandria Engineering Journal*, 1-10.
- Mulla, F. (2024). Choosing the Best Architecture for Mobile Applications. *ResearchGate*.
- Sahu, D., et al. (2025). Edge assisted energy optimization for mobile AR applications. *Scientific Reports*.